Homework 01

Question 1 (Least squares and matrix calculations)

Solution:

```
# Question 1: Matrix Calculations
set.seed(43201)
n <- 100
p <- 5
X_preds <- matrix(rnorm(n * p), n, p)
X <- cbind(1, X_preds)
beta <- c(3, 1.4, -0.9, 0.7, 0, 1.1)
y <- X %*% beta + rnorm(n, 0, 0.7)

# 1a: Dimensions, rank, and stable estimation
cat("Dimensions of X:\n")
```

```
Dimensions of X:
```

```
print(dim(X))
```

```
[1] 100   6
```

```
cat("Rank of X:", qr(X)$rank, "\n")
```

```
Rank of X: 6
```

```r
beta_hat <- qr.coef(qr(X), y)

# Side-by-side comparison of true beta vs estimated beta_hat
comparison <- data.frame(
  Term       = c("Intercept", "x1", "x2", "x3", "x4", "x5"),
  True_beta  = beta,
  Beta_hat   = round(beta_hat, 4),
  Difference = round(beta_hat - beta, 4)
)
cat("\nComparison of true coefficients vs. least-squares estimates:\n")
```

```
Comparison of true coefficients vs. least-squares estimates:
```

```r
print(comparison, row.names = FALSE)
```

```
      Term True_beta Beta_hat Difference
 Intercept       3.0   3.0063     0.0063
        x1       1.4   1.4172     0.0172
        x2      -0.9  -0.8883     0.0117
        x3       0.7   0.7953     0.0953
        x4       0.0   0.0484     0.0484
        x5       1.1   1.1319     0.0319
```

Solution 1b:

```r
# Calculate fitted values
fitted_values <- X %*% beta_hat

# Calculate the residual vector
r <- y - fitted_values

# Calculate the training root mean squared error (RMSE)
rmse <- sqrt(mean(r^2))

cat("Training RMSE:", rmse, "\n")
```

```
Training RMSE: 0.6763696
```

1c Solution:

```
# Calculate the infinity norm of X^T r (maximum absolute value)
XTr <- t(X) %*% r
inf_norm_XTr <- max(abs(XTr))

cat("Infinity norm of X^T r:", inf_norm_XTr, "\n")
```

```
Infinity norm of X^T r: 8.915091e-14
```

2a Solution:

The log-likelihood function that is provided is:

$$\ell(\beta, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\beta)^\mathsf{T}(\mathbf{y} - \mathbf{X}\beta)$$

The first term $-\frac{n}{2}\log(2\pi\sigma^2)$ is a constant with regard to β for a fixed σ^2. The residual sum of squares (RSS) is precisely contained in the second term, $(\mathbf{y} - \mathbf{X}\beta)^\mathsf{T}(\mathbf{y} - \mathbf{X}\beta)$. The precisely negative number $-\frac{1}{2\sigma^2}$ is multiplied by this term. Therefore, we need to minimize the RSS in order to maximize ℓ. This suggests that the ordinary least-squares estimate and the maximum-likelihood estimate of β are the same.

2b Solution:

Differentiate with regard to σ^2 and put equal to zero to determine the MLE of σ^2. First, differentiate:

$$\frac{\partial}{\partial(\sigma^2)}\ell(\beta, \sigma^2) = -\frac{n}{2\sigma^2} + \frac{(\mathbf{y} - \mathbf{X}\beta)^\mathsf{T}(\mathbf{y} - \mathbf{X}\beta)}{2(\sigma^2)^2}$$

Step 2: Set the value to zero and solve:

$$0 = -n\sigma^2 + (\mathbf{y} - \mathbf{X}\beta)^\mathsf{T}(\mathbf{y} - \mathbf{X}\beta)$$

$$\hat{\sigma}^2_{\text{MLE}} = \frac{1}{n}(\mathbf{y} - \mathbf{X}\hat{\beta})^\mathsf{T}(\mathbf{y} - \mathbf{X}\hat{\beta}) = \frac{1}{n}\sum_{i=1}^{n} r_i^2$$

```r
# 2b: MLE and unbiased estimate of sigma^2

# Calculate residuals at beta_hat
r <- y - X %*% beta_hat

# Residual sum of squares
RSS <- sum(r^2)

# MLE of sigma^2: divides by n
sigma2_mle <- RSS / n

# Unbiased estimate of sigma^2: divides by n - p_cols (degrees of freedom)
p_cols <- ncol(X)
sigma2_unbiased <- RSS / (n - p_cols)

cat("MLE of sigma^2 (denominator n)      :", sigma2_mle, "\n")
```

```
MLE of sigma^2 (denominator n)      : 0.4574759
```

```r
cat("Unbiased estimate (denominator n-6) :", sigma2_unbiased, "\n")
```

```
Unbiased estimate (denominator n-6) : 0.4866765
```

2c Solution:

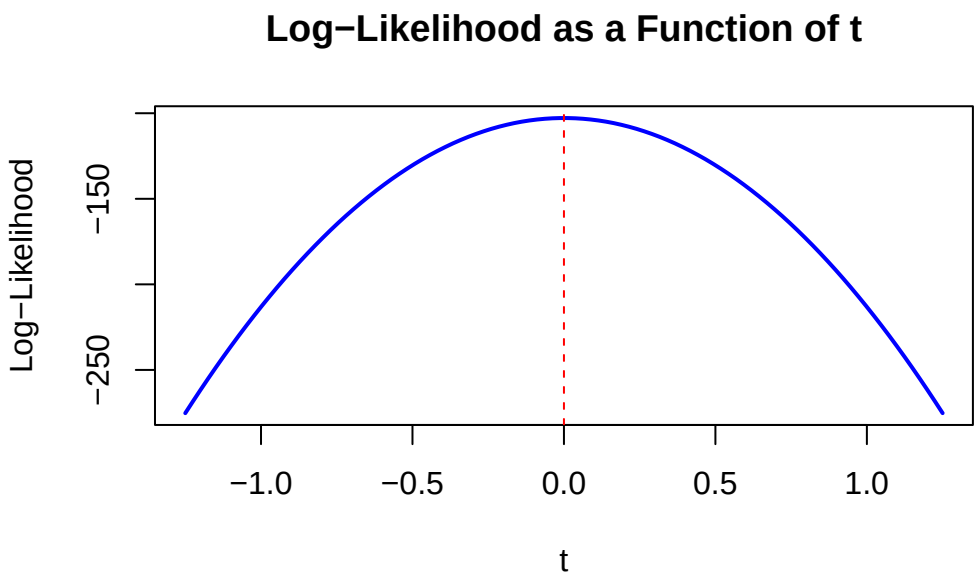
```r
e_x2   <- c(0, 0, 1, 0, 0, 0)
t_vals <- seq(-1.25, 1.25, length.out = 100)

calc_ll <- function(t) {
  beta_t <- beta_hat + t * e_x2
  r_t    <- y - X %*% beta_t
  rss_t  <- sum(r_t^2)
  -(n / 2) * log(2 * pi * sigma2_mle) - (1 / (2 * sigma2_mle)) * rss_t
}

ll_vals <- sapply(t_vals, calc_ll)

plot(t_vals, ll_vals, type = "l", col = "blue", lwd = 2,
     xlab = "t", ylab = "Log-Likelihood",
```

```
     main = "Log-Likelihood as a Function of t")
abline(v = 0, col = "red", lty = 2)
```




```
max_t <- t_vals[which.max(ll_vals)]
cat("The maximum log-likelihood occurs at t =", max_t, "\n")
```

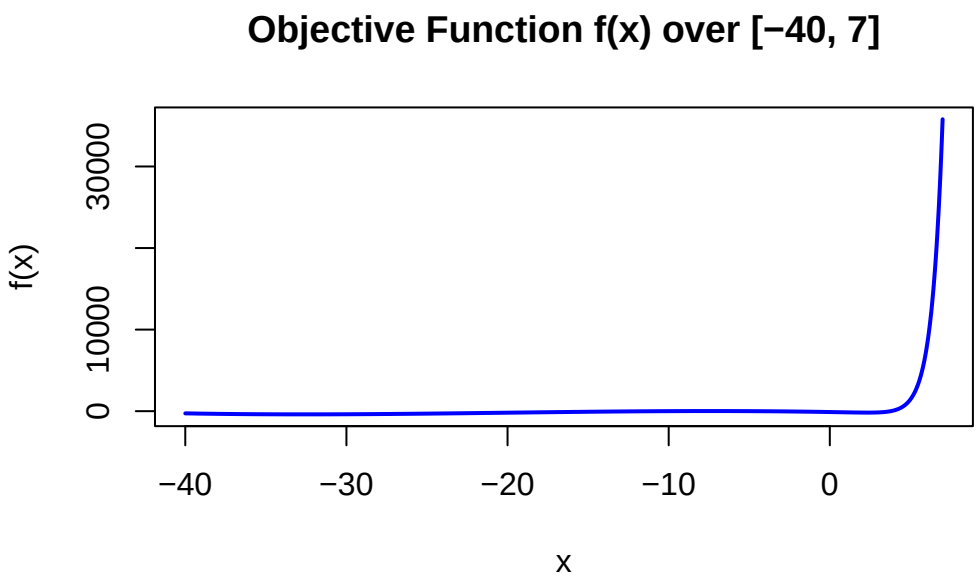
```
The maximum log-likelihood occurs at t = -0.01262626
```

Question 3 (Starting values in nonconvex optimization)

3a Solution:

```
f       <- function(x) exp(1.5 * x) - 3 * (x + 6)^2 - 0.05 * x^3
f_prime <- function(x) 1.5 * exp(1.5 * x) - 6 * (x + 6) - 0.15 * x^2

curve(f, from = -40, to = 7, n = 1000, col = "blue", lwd = 2,
      ylab = "f(x)", main = "Objective Function f(x) over [-40, 7]")
```



##3b Solution:

```
run1 <- optim(par = -15, fn = f, gr = f_prime, method = "BFGS")
run2 <- optim(par = 0,   fn = f, gr = f_prime, method = "BFGS")

results <- data.frame(
  Start_Val     = c(-15, 0),
  Final_x       = c(run1$par, run2$par),
  Objective_Val = c(run1$value, run2$value),
  Abs_Gradient  = c(abs(f_prime(run1$par)), abs(f_prime(run2$par))),
  Converged     = c(run1$convergence == 0, run2$convergence == 0)
)
print(results)
```

```
  Start_Val    Final_x Objective_Val Abs_Gradient Converged
1       -15 -32.649111     -390.3858 8.725087e-08      TRUE
2         0   2.349967     -175.8626 6.556057e-07      TRUE
```

3c Solution:

Because $f(x)$ is non-convex with multiple local minima, the BFGS algorithm simply gets trapped in whichever valley is closest to its starting point. Initializing at $x^{(0)} = -15$ forces it

into a left-side valley, while starting at $x^{(0)} = 0$ directs it into a completely different right-side valley.The run starting at $x^{(0)} = -15$ finds the lower objective value (-390.39 compared to -175.86 from the $x^{(0)} = 0$ start).

Question 4 (Nearly collinear predictors)

4a Solution:

```
set.seed(43202)
u      <- rnorm(n)
x6     <- X[, 2] + 1e-4 * u
X_plus <- cbind(X, x6)
y_star <- y + 1e-3 * u

beta_X      <- qr.coef(qr(X), y)
rmse_X      <- sqrt(mean((y - X %*% beta_X)^2))
cond_X      <- kappa(X, exact = TRUE)

beta_X_plus <- qr.coef(qr(X_plus), y)
rmse_X_plus <- sqrt(mean((y - X_plus %*% beta_X_plus)^2))
cond_X_plus <- kappa(X_plus, exact = TRUE)

cat("Condition number of X      :", cond_X, "\n")
```

```
Condition number of X      : 1.524914
```

```
cat("Condition number of X_plus :", cond_X_plus, "\n")
```

```
Condition number of X_plus : 20209.2
```

```
cat("RMSE of X fit              :", rmse_X, "\n")
```

```
RMSE of X fit              : 0.6763696
```

```
cat("RMSE of X_plus fit         :", rmse_X_plus, "\n")
```

```
RMSE of X_plus fit         : 0.6754668
```

```r
cat("Coefficient of x1 (X_plus) :", beta_X_plus[2], "\n")
```

```
Coefficient of x1 (X_plus) : 351.6766
```

```r
cat("Coefficient of x6 (X_plus) :", beta_X_plus[7], "\n")
```

```
Coefficient of x6 (X_plus) : -350.2588
```

4b Solution:

The changes in the coefficients are obtained by comparing the fit of $\mathbf{y}^\star$ against that of $\mathbf{y}$, both using $\mathbf{X}_+$.

```r
beta_star <- qr.coef(qr(X_plus), y_star)

cat("Change in x1 coefficient   :", beta_star[2] - beta_X_plus[2], "\n")
```

```
Change in x1 coefficient   : -10
```

```r
cat("Change in x6 coefficient   :", beta_star[7] - beta_X_plus[7], "\n")
```

```
Change in x6 coefficient   : 10
```

```r
fit_y      <- X_plus %*% beta_X_plus
fit_y_star <- X_plus %*% beta_star

rmse_diff    <- sqrt(mean((fit_y - fit_y_star)^2))
max_abs_diff <- max(abs(fit_y - fit_y_star))

cat("RMSE diff in fitted values :", rmse_diff, "\n")
```

```
RMSE diff in fitted values : 0.001015879
```

```r
cat("Max abs diff in fits       :", max_abs_diff, "\n")
```

```
Max abs diff in fits       : 0.002572675
```

4c Solution:

The exact difference between the two response variables is $\mathbf{y}^\star - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1)$. Because $\mathbf{x}_6$ and $\mathbf{x}_1$ are nearly identical, this difference perfectly equals the tiny noise vector $(10^{-3}\mathbf{u})$ added to the data.
To account for this tiny shift, the model drastically adjusts the individual coefficients, shifting $\mathbf{x}_1$ by -10 and $\mathbf{x}_6$ by $+10$. Because the variables are so similar, these massive, opposing changes cancel each other out. As a result, the total combined prediction remains almost exactly the same (the fitted values moved by only ≈ 0.001).

Question 5 (Data preparation and summary statistics)

5a Solution:

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
df <- read.csv("data/data-manipulation.csv")

cat("Dimensions of the dataset:\n")
```

```
Dimensions of the dataset:
```

```
print(dim(df))
```

```
[1] 24  5
```

```r
cat("\nColumn types:\n")
```

```
Column types:
```

```r
print(sapply(df, class))
```

```
observation_id          group             x1             x2       response
   "character"    "character"      "numeric"      "numeric"      "numeric"
```

```r
num_dups <- sum(duplicated(df$observation_id))
cat("\nNumber of duplicated identifiers:", num_dups, "\n")
```

```
Number of duplicated identifiers: 0
```

```r
cat("\nNumber of missing values per column:\n")
```

```
Number of missing values per column:
```

```r
print(colSums(is.na(df)))
```

```
observation_id          group             x1             x2       response
             0              0              0              0              3
```

```r
analysis_df <- df %>%
  filter(!is.na(response)) %>%
  mutate(feature_sum = x1 + x2)
```

5b Solution:

These summaries describe only the 21 observations with a recorded (non-missing) response — the rows retained in `analysis_df` after the 3 missing-response rows were dropped from the original 24. Observations with a missing response are excluded entirely.

```r
group_summaries <- analysis_df %>%
  group_by(group) %>%
  summarize(
    n_observations   = n(),
    mean_response    = mean(response),
    mean_feature_sum = mean(feature_sum)
  )

cat("Group summaries (21 observations with non-missing response):\n")
```

```
Group summaries (21 observations with non-missing response):
```

```r
print(group_summaries)
```

```
# A tibble: 3 x 4
  group n_observations mean_response mean_feature_sum
  <chr>          <int>         <dbl>            <dbl>
1 A                  7          5.6              3.6
2 B                  7          7.60             4.93
3 C                  7          8.00             6.59
```

5c Solution:

```r
top_3_responses <- analysis_df %>%
  arrange(desc(response)) %>%
  select(observation_id, group, response, feature_sum) %>%
  head(3)

cat("\nFirst three rows sorted by response:\n")
```

```
First three rows sorted by response:
```

```r
print(top_3_responses)
```

```
  observation_id group response feature_sum
1         obs-22     C     9.24         6.6
2         obs-16     B     9.00         5.2
3         obs-14     B     8.66         5.0
```

```r
stopifnot(nrow(analysis_df) == sum(!is.na(df$response)))
stopifnot(all(!is.na(analysis_df$response)))
stopifnot(anyDuplicated(analysis_df$observation_id) == 0)

cat("\nAll code checks passed successfully!\n")
```

```
All code checks passed successfully!
```

Question 6 (Can 39 million Fitbit records represent US adults?)

6a Solution:

The 39 million daily records are not independent observations because they come from a finite number of participants, each contributing multiple days of data. Characteristics or behaviors that could cause some participants to contribute more recorded days than others include personal motivation, health status, lifestyle, and adherence to wearing the Fitbit device. For example, more active individuals or those with a strong interest in tracking their fitness may log more days, while less active or less engaged participants may log fewer days. As a result, averaging all recorded days would give disproportionate weight to participants who contributed more data, leading to an estimate that reflects the behavior of these more active individuals rather than the average behavior of the entire US adult population.

6b Solution:

The comparison does not show that owning a Fitbit causes people to walk more. This is an observational comparison, not a randomized experiment, so confounding is plausible. BYOD participants self-selected into Fitbit ownership before the study, which likely correlates with being more health-conscious, more physically active, and having higher socioeconomic status — all independently associated with higher step counts. The demographic differences confirm this: BYOD participants are substantially more likely to be White and less likely to be in a low-income bracket than WEAR participants. The observed step-count gap could therefore reflect these pre-existing differences rather than any effect of Fitbit ownership itself.

6c Solution :

There are two distinct layers of selection bias that together make the naive daily average a poor estimator of the target (equal-weighted mean of participant-specific average daily steps among US adults).

Participant-level selection. All of Us participants are not a random sample of US adults. BYOD enrollees self-selected by already owning a Fitbit, which correlates with higher activity, higher income, and higher health engagement — as confirmed by the demographic differences reported in the study. WEAR participants are more diverse but still not population-representative, as they were recruited through a research program that itself attracts people with specific interests.

Day-level selection within participants. Even among enrolled participants, the number of days contributed is not random. People who are more active, healthier, or more engaged with their device tend to wear it on more days. Together, these mechanisms mean that the 39 million records — despite their volume — cannot be treated as a representative sample of US adult daily activity.